DayJournal design

This is a map of the source for someone about to change it. The user manual says what the program does; this says where and how. DayJournal is written in Python with PySide6 (Qt 6), like pylg, and uses easyocr for the OCR.

The pieces

File	What is in it
`dj.py`	`main`: starts the window, or the OCR process with `--ocr`
`mainwin.py`	`MainWindow`: actions, menus, the list, search, calendar, tags, notes, saving
`journal.py`	`Journal` and `Entry`: the directory, names and dates, notes, drawings, the tags cache. No widgets
`tags.py`	`cleantags`: from OCR text to tags
`ocr.py`	`readimage` with easyocr, and `serve`, the loop of the OCR process
`ocrworker.py`	`OCRWorker`: runs the OCR process with `QProcess` and reads its answers
`capture.py`	adding a capture: `guess` (its date from the OCR, its file name and date), `CaptureDialog`, `savecapture`
`search.py`	`Search`: parses and evaluates search expressions
`viewer.py`	`ImageView`: a `QGraphicsView` with the image, the zoom and the drawing tools
`drawing.py`	the shapes (`LineShape`, `PencilShape`, `CircleShape`, `RectShape`, `TextShape`, `FibShape`), the fib level sets, their JSON form and the undo commands
`drawbar.py`	`DrawBar`: the toolbar for an `ImageView` (tools, color, width, undo, redo, delete)
`tests/`	headless tests; the OCR is faked
`dj.spec`	pyinstaller spec: `DayJournal.app` on macOS

`drawing.py`, `drawbar.py` and the drawing part of `viewer.py` are the same code used in pylg's graphics window; keep them in step when changing one.

Data on disk

Everything is in the journal directory, so it can be copied, backed up or synced as a whole. Journals are looked for as `DayJournal*` in `journal.tradedirs()`: `/u/trade`, or `~/u/trade`, and when there is no `u` directory (`UDIRS`), `trade` in the system's documents directory (`QStandardPaths`) and the documents directory itself. `tradedir()` is where **Open** starts when there is no journal yet. The recent journals (`recentjournals`, `addrecent` in `mainwin.py`) leave out the ones in temporary directories (`istemp`, `TEMPDIRS`).

File	Written by	Contents
`dayYYMMDD[x].png`, `dayYYMMDDweekprof.png`	you	the images; DayJournal never changes them
`<image>.txt`	`Journal.savenotes`	the notes, plain UTF-8 text
`<image>.drw`	`Journal.savedrawing`	the drawings, JSON (below)
`.pyjou-tags.json`	`Journal.savecache`	`{"version": 1, "images": {name: {"mtime", "raw": [[text, confidence]...]}}}`

All writes go to a temporary file first and then `os.replace` it, so a crash never leaves half a file. An empty notes or drawing file is removed instead of written.

The tags cache keeps the raw OCR text, not the tags, so changing the rules in `tags.py` takes effect at once without reading the images again. An image is read again when its modification time changes.

A drawing file:

```
        {"version": 1, "shapes": [
          {"kind": "arrow", "p1": [1167.4, 1307.5], "p2": [1494.3, 1097.3], "color":
"#e53935", "width": 3.0},
          {"kind": "circle", "rect": [1400.9, 1004.0, 233.5, 140.1], "color": "#e53935",
"width": 3.0},
          {"kind": "pencil", "points": [[10, 20], [12, 24]], "color": "#2e7d32", "width":
3.0},
          {"kind": "text", "pos": [800, 150], "text": "EQH sweep", "color": "#1e63d6",
"size": 22}
        ]}
```

Coordinates are image pixels, so the drawing does not depend on the zoom. `line` is like `arrow`. Unknown kinds are skipped when reading.

Names and dates

`journal.parsename` turns a file name into an `Entry` (date, week profile or not, the extra letter). Weeks start on Monday (`weekstart`). A week profile dated a Sunday is for the week starting the next day; any other one is for the week of its date (`Entry.weekmonday`). `Entry.groupday` is the date an entry is placed at: its own date for a day, the Monday of its week for a profile. `sortkey`, `groupkeys` (year, month and week groups of the list), `Journal.weekof` and `Journal.daysof` all build on these. `Journal.weeks` are the Mondays of the weeks with images, and `weekno(monday, first)` numbers a week in calendar weeks from the first one (1); `fillist` uses them for the week labels and the count. `MainWindow.collapseto` is the **m** and **w** buttons.

The window

`MainWindow` keeps:

- `journal`, the open `Journal`, and `current`, the `Entry` shown;
- `shown`, the entries that pass the search and the Days/Weeks boxes, in order. Moving to the previous or next image, the calendar and First/Last all go through `shown`;
- `items` and `groups`, the tree items of the list, and `expanded`, the keys of the groups open, which survive refilling the list;
- `pending`, the drawings changed and not saved, by image name.

New Window (`newwindow`) makes another `MainWindow` on the same journal, showing the same image with the same unsaved drawings; the extra windows are kept in `mainwin.windows` until closed. Windows share nothing in memory: each has its own `Journal`, and `reload` (run by the directory watcher) shows notes and drawings that another window saved for the image shown.

`show_(e)` is the one place an image is shown: it saves the notes, ends the editing of a text box, and loads the image, its notes, tags and drawing (`drawingof`: the pending one, or the saved one).

`applysearch` recomputes `shown` and refills the list (`fillist`). It runs when the search text, the Days/Weeks boxes, or the tags change.

The left panel (`left`) and the tags and notes (`bottom`) are hidden and shown by `showpanel`, kept in the settings (`showlist`, `shownotes`).

A `QFileSystemWatcher` on the directory calls `reload` a second after a change: it reads the directory again and shows the current image again only if its file changed, so saving the notes doesn't reset the drawing or its undo history.

OCR

Running easyocr (torch) in a thread of the Qt program deadlocked, and importing it there is very slow under PySide's import hook. So the OCR runs in a separate process: `ocrworker.OCRWorker` starts this same program with `--ocr` (`sys.executable` with `dj.py`, or the app itself when built by pyinstaller), writes the image paths to its standard input, one per line, and reads one JSON line per image back: `{"path", "raw"}` or `{"path", "error"}`. `dj.py` handles `--ocr` before importing Qt.

`MainWindow.startocr` sends the images whose cache entry is missing or old, newest first; `tagged` stores each result, saves the cache every few images, and refreshes the tags and the search. Images asked for while the process runs wait in `ocrpending`.

`tags.cleantags` keeps the words that look like words: it splits on punctuation, drops low confidence text, numbers, dates, month and weekday names, short Spanish and English words, chart chrome (`NOISE`), lowercase runs without vowels and misread capitals (`PtHOG`), and keeps time frames such as `m15`.

Adding captures

`MainWindow` accepts drops of image files (the view and the notes don't, so they reach the window) and **Add Capture...** picks them with a file dialog; both end in `addcaptures`. For each file, `CaptureDialog` starts its own `OCRWorker` on it and shows `capture.guess` at once with the file's date, then again when the OCR answers, unless the user changed the date or the week box meanwhile.

`guess` takes the dates in the OCR text (`datesin`: `YY-MM-DD` as the charts write them, `YYYY-MM-DD`, `D/M/Y`, `14 oct 2026`, `Oct 14`; `mar` is not taken as March before a number, since it is Tuesday in Spanish), in the order the OCR found them, and adds the day the file was last changed. The guess is the last date in the image not after the file's day (charts show the days before), or the file's day when there is none; but a Sunday found with the Friday after it wins, as a week profile. `kindof` makes a weekend day a week profile, named on its Sunday (`weekname`), and any other day a day; picking a date in the dialog uses it too.

A double click on an image of the list opens the same dialog with `entry` (`MainWindow.changedate`): it starts with the entry's date and kind, takes the OCR text from the cache instead of reading the image, and `newname(..., keep=entry)` keeps the entry's own name when its date and kind don't change. `Journal.rename` moves the image, its `.txt` and `.drw` files and its cache entry, refusing to replace any file.

`Journal.newname` gives the name: the plain one for the first capture of a day (or week profile), then the letter after the last one there, counting the plain one as `a`. `savecapture` copies through a temporary file, or converts to PNG with `QImage`, and never replaces a file. The OCR results are stored in the tags cache for the new name, so it is not read again.

Cut, copy and paste in the list (`MainWindow.clipcapture` and `pastecapture`, in the list's context menu) put the journal directory, the image name and whether it was cut on the system clipboard as JSON, with the `CLIPMIME` type, so another DayJournal process can paste it too; the image's file URL goes with it for other programs. `Journal.paste(src, entry, move)` copies the image, its `.txt` and `.drw` and its cache entry from journal `src` under `newname` for the entry's date and kind, removing them from `src` for a cut. A cut is pasted only once (the clipboard is cleared) and never in its own journal. Clipboard files without that type go to `addcaptures`.

Search

`search.Search` is a small recursive descent parser:

```
or   := and ('|' and)*
and  := not+
not  := '!' not | '(' or ')' | word
```

Each word becomes a regex (`wordpattern`): a whole word, ignoring case, with an optional lowercase final `s` and `*` as any word characters. An entry matches when the expression is true over its tags, notes and the text of its drawing (`Entry.drawtext`). A bad expression raises `ValueError`; the search box shows the message and keeps the last good search.

Windows

`winlayout.py` (the same file in RoadBook and DayJournal; keep them the same) makes the Window menu (`windowmenu`). `windows()` are the visible top level windows of kind Window or Dialog (not tool windows, so not the floating draw bar or docks), sorted by their place on the screen with the active one first. The layouts in `ALL` take the screen rectangle and the number of windows and return a rectangle for each (`grid`, `split`, `mainleft`, `maintop`, `cascade`); those in `ONE` place the active window alone. `place` sets a window's frame on its rectangle and moves it back on the screen when its

minimum size is larger. `narrow` lets push buttons shrink to their text, so the windows fit those rectangles. `MainWindow.mkmenus` adds it, and `dupicon` paints the New Window icon.

Drawing

`ImageView` puts the image in a `QGraphicsScene` as a `QGraphicsPixmapItem` at z 0, and each drawing as an item above it, all in image coordinates. Zooming changes only the view transform. `viewportEvent` takes the trackpad gestures: a two finger double tap (`SmartZoomNativeGesture`) steps the zoom (`zoomstep`: in, in again, fit) and pinching zooms. Double clicks on the image step it too when `dclickzoom`, which DayJournal turns off.

- The tool (`settool`) decides what a mouse press does: with **select** the scene handles it (selecting and moving items), and a drag on the image pans (`ScrollHandDrag`); with a drawing tool the press creates a shape and the moves call its `setend` until the release. A fib takes more steps (`fibstep`: `drag`, then `p2` when the first press was a click, then `right`, following the mouse with no button down); the click that sets the right end finishes it after the double click interval (`fibdone`), unless a double click comes, which takes the lines to the image's right edge (`extend`); the right button or Esc drops it (`cancelfib`). While it is drawn it is `live`, with a dashed line at `p2`. Its JSON has `p1`, `p2`, `right`, `extend`, `diagonal` and `levels`; level 0 is at `p2` and 1 at `p1`. `DrawBar` keeps the 20 rows of the custom form (`FibDialog`: on, text) and its diagonal as JSON in `QSettings("fjballest", "drawing")`, key `customfibrows`, shared by both programs.
- `DrawBar.detach` moves the bar into a `FloatBar`, a tool window that stays on top, remembering where it was (the `QMainWindow` toolbar area, or the layout and index); `attach`, or closing the window, puts it back. Undo and redo are added to the main window so their keys still work.
- Each change is a `QUndoCommand` on the view's `QUndoStack`: `AddCmd`, `DeleteCmd`, `MoveCmd` (made on release from the positions at the press), `TextCmd`, `StyleCmd`, `LevelsCmd` (fib levels) and `PropsCmd` (a rectangle's fill, middle line and label and their colors, set in `RectDialog`, opened by a double click on it, or by a preset of `drawing.RECTSETS`, picked in the Rect menu of `DrawBar` and kept in `ImageView.rectpreset` for the next rectangles). Commands made after the fact (the shape is already drawn) skip their first `redo`.
- Every shape but the text boxes has a `dash` (solid, dash, dot); lines, arrows, circles and rectangles a `label` with its `size` and `labelcolor`, placed by `labelrect` (lines and circles also by `valign` and `align`; a label past the right end is kept on the image by `inscene`, and `clipout` keeps the line from crossing it). `PROPS` lists what `setprops` changes; `PropsCmd` undoes it. A double click on a line, arrow, circle or rectangle calls `ImageView.editshape`, which opens `ShapeDialog` or `RectDialog` and applies their `changed` at once, putting it back on Cancel. The bar's style and text size apply through `setselprops` to what is selected, in one undo step; **Clear** is a `DeleteCmd` of every shape. The fib numbers go past the end of each line, and lines taken to the edge stop short to leave them room (`xs`).
- A text box is edited in place; when it loses the focus it calls `ImageView.textedited`, which makes the undo step, or removes it if empty.
- Keys on the view: Del deletes, Esc deselects, S/P/L/A/C/R/T/F pick the tool, and the arrows, Home, End and W (`navkeys`) are sent as `navigate` to the window, unless a text box is being edited.
- `drawchanged` is emitted on every undo stack change. The window then compares `drawingdata()` with the saved drawing and keeps it in `pending` if they differ; the title says

(unsaved) while `pending` has anything. **Save** writes them all (`savedrawings`); quitting or opening another journal asks first (`keepdrawings`).

The undo stack is cleared when another image is shown, so undo is per image and per visit.

Tests

```
QT_QPA_PLATFORM=offscreen python3 tests/test_journal.py
QT_QPA_PLATFORM=offscreen python3 tests/test_drawing.py
QT_QPA_PLATFORM=offscreen python3 tests/test_capture.py
QT_QPA_PLATFORM=offscreen python3 tests/test_clip.py
QT_QPA_PLATFORM=offscreen python3 tests/test_panels.py
QT_QPA_PLATFORM=offscreen python3 tests/test_window.py
QT_QPA_PLATFORM=offscreen python3 tests/test_layout.py
```

`tests/sandbox.py` makes them keep their settings in files of their own, since on macOS `QSettings` ignores `HOME` and would use the user's.

They run without a display and without easyocr: `ocrworker.ocrcommand` is replaced by a small script that answers with made up text. `python3 ocr.py image.png` prints what the real OCR reads in an image and the tags made from it, to tune `tags.py`.

Building

`pyinstaller --noconfirm dj.spec` makes `dist/DayJournal.app` on macOS (and `dist/dj` elsewhere). The workflow in `.github/workflows` builds it on GitHub for the three systems. The OCR models are not bundled: easyocr keeps them in `~/.EasyOCR` and downloads them the first time.

The PDFs next to these documents are made with `docs/mkpdf.sh`.

Where to change things

To	Change
accept other image names	`NAMERE` and `parsename` in `journal.py`, `capturename`
find other ways of writing dates	`datesin` in `capture.py`
keep or drop other words as tags	`tags.py` (`NOISE`, `STOP`, `isword`)
change how searches match	`search.py` (`wordpattern`, the parser)
add a kind of drawing	a shape class with `todict`, `setend`, `small` and `setstyle` in `drawing.py`, `fromdict`, the tool in `ImageView.mousePressEvent` and `drawbar.TOOLS`
add a menu entry	`MainWindow.mkactions` and `mkmenus`
change the list groups	`journal.groupkeys` and `MainWindow.initialgroups`